

AERIS

AI-ASSISTED AUTONOMOUS DISASTER RESPONSE & EMERGENCY LOGISTICS UAV

Prototype Master Build Specification

Revision 2.0 | Engineering-Frozen Physical Prototype Specification

Prepared 26 September 2026

This specification consolidates the supplied AERIS concept, Robofest application, application forms, and final hackathon build plan, then applies the engineering refinements identified during validation.

AERIS Prototype - End-to-End System Architecture

Physical F450 prototype with edge AI, autonomous flight, human-supervised response and local ground station

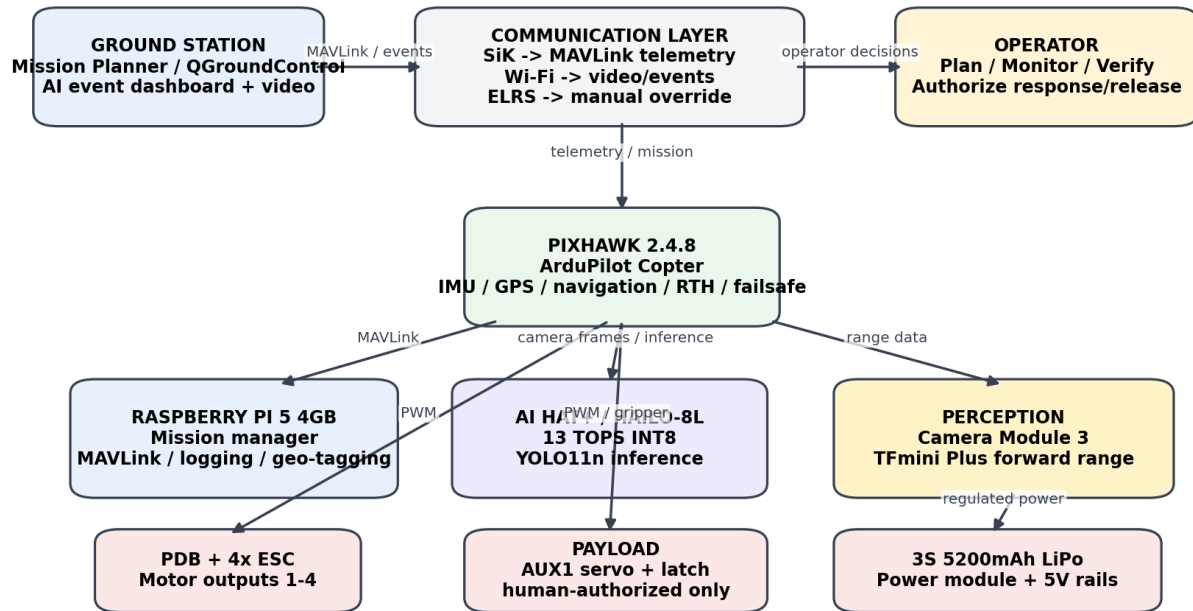


Figure 1. Final prototype system architecture.

PROTOTYPE BOUNDARY: The physical aircraft is an F450-class quadcopter. The 5-kg emergency-logistics platform is a digital-twin target only. The physical payload is a small demonstration payload and is permitted only after measured thrust margin and flight validation.

SAFETY BOUNDARY: AERIS is a research/hackathon prototype, not a production search-and-rescue aircraft. Human supervision remains mandatory for consequential response and payload release.

Document Control and Engineering Freeze

Field	Frozen value
Project	AERIS - AI-Assisted Autonomous Disaster Response & Emergency Logistics UAV
Prototype vehicle	F450 X-configuration quadcopter
Revision	2.0 - prototype engineering freeze
Date	26 September 2026
Primary flight controller	Pixhawk 2.4.8 / FMUv2-class, ArduPilot Copter
Companion computer	Raspberry Pi 5, 4GB
AI accelerator	Raspberry Pi AI HAT+ 13 TOPS, Hailo-8L
Primary AI model	Custom YOLO11n detector exported as Hailo HEF
Ground station	Mission Planner primary; QGroundControl backup; local AI/video dashboard
Communication	SiK telemetry for MAVLink; Wi-Fi for local video/events; ELRS for manual override
Payload	Human-authorized servo gripper/drop latch; small demonstration load only
Digital twin	Gazebo + ArduPilot SITL, conceptual 5-kg hexacopter only

Change Log From the Supplied Build Plan

Change	Reason	Disposition
Corrected Quad-X motor map	Original conceptual drawing was ambiguous versus standard ArduPilot mapping.	Frozen in Sections 9 and 15.
Dedicated Pi 5 5.1V >= 5A rail	Generic BEC wording was insufficient for Pi 5 + AI HAT+ operation.	Frozen in power tree and BOM.
Dedicated servo BEC	Do not depend on flight-controller rail for the payload servo.	Frozen in Section 15.
Explicit UART plan	GPS, SiK, ELRS, TFmini and companion link must coexist.	Frozen in pin map.
USB-UART for Pi <-> Pixhawk	Separates Pi mission link from TFmini UART and avoids interface contention.	Frozen as 3.3V-logic USB-UART.
Wi-Fi video path	SiK is treated as MAVLink telemetry, not video transport.	Frozen in communication architecture.
YOLO11n baseline	Provides a small current Hailo-compatible detector; custom classes are trained.	Frozen baseline; exact trained weights are a build artifact.
ByteTrack + event de-duplication	Prevents repeated alerts from the same target across frames.	Frozen.
Ground-projection geo-tagging	Drone GPS alone is aircraft position, not target position.	Frozen as approximate target geolocation with fallback.
Forward-only obstacle behavior	One TFmini cannot provide 360-degree perception.	Frozen as warning/hold first; reroute is optional after validation.
Explicit mission state machine	Turns features into deterministic, testable mission behavior.	Frozen in Section 13.
Budget normalization	Multiple supplied documents used different budget figures.	Planning budget is presented as a refined engineering estimate with sourcing caveat.

Source Basis

The system architecture and naming are derived from the supplied AERIS Concept & Prototype document, Robofest technical application, R620260083 application materials, AERIS GUJCOST technical details, and the Final Hackathon Build Plan. The technical validation layer additionally uses current official documentation for Raspberry Pi, ArduPilot, Benewake, Hailo/Ultralytics, and selected research literature.

Contents

1. Idea, Mission and Societal Application
2. Prototype Scope and Capability Boundary
3. System Requirements and Success Criteria
4. Final Hardware Bill of Materials
5. Hardware Technical Design
6. Power, Weight, Thrust and Energy Engineering
7. Final Software Technical Stack
8. AI Model, Data, Mathematics and Algorithms
9. Full Hardware-Software Architecture
10. Electrical and Communication Interface Specification
11. Pin-by-Pin Wiring and Connection Tables
12. Software Repository and Runtime Structure
13. Mission State Machine and Decision Logic
14. Full Implementation Roadmap
15. Hardware Build Procedure
16. Software Build and Integration Procedure
17. Integration, Calibration and Validation Gates
18. End-to-End Working of the Built Prototype
19. Digital Twin / SITL Implementation
20. Test Cases, Metrics, Logging and Acceptance
21. Calculations and Engineering Checks
22. Failure Modes and Recovery Logic
23. Deployment/Demo Procedure
24. Known Limits and Remaining Measured Quantities
25. Final Validation and Summary
26. Technical References

NAVIGATION: The document is intentionally organized as an engineering build specification. Hardware interfaces are defined before the implementation sections so that the software is implemented against stable physical interfaces.

1. Idea, Mission and Societal Application

AERIS is an AI-assisted autonomous disaster-response UAV intended to improve the speed and quality of initial aerial reconnaissance. The core idea is not simply to fly a camera over a disaster zone. AERIS turns aerial observations into structured, time-stamped and location-associated incident information, presents that information to a human operator, and then connects a verified incident to a supervised response mission. This combines autonomous flight, edge AI, forward obstacle awareness, localization, event prioritization, telemetry, operator supervision and an optional small payload release into one closed-loop prototype.

The supplied concept document describes use in floods, earthquakes, landslides, cyclones or severe storms, urban emergencies and remote/hard-to-reach areas. The same architecture also applies to post-disaster inspection, missing-person search support, blocked-road reconnaissance, visible-debris surveys and rapid situational awareness where ground access is difficult [D1].

1.1 The problem

During a disaster, responders may have incomplete situational awareness, unsafe or blocked access routes, and a large area to inspect. Aerial platforms can cover ground faster than a person walking the same area, but a manually piloted camera drone still leaves the operator to watch the video continuously, notice important events, estimate locations, decide whether observations are actionable, and coordinate the response. AERIS adds machine perception and structured incident handling while keeping the operator responsible for consequential actions.

1.2 The proposed response loop

Mission definition -> autonomous takeoff -> coverage search -> camera capture -> edge AI -> detection
-> tracking / confirmation -> geo-location -> priority -> ground alert -> human verification
-> response waypoint -> supervised payload release (optional) -> return or resume search

1.3 What makes the prototype useful

- It reduces the cognitive burden of continuously inspecting aerial video by producing explicit detection events rather than relying only on a video stream.
- It preserves human authority. The AI proposes an incident; the operator verifies it before a consequential response action.
- It provides a repeatable search method through grid/lawnmower coverage instead of ad-hoc manual flight.
- It creates a machine-readable incident record that can be reviewed after the mission, replayed in the digital twin, and used to compare field runs.
- It separates safety-critical stabilization and failsafes from best-effort AI so an AI software fault is not required for the aircraft to remain flyable.
- It is intentionally scalable: the perception and mission-management interfaces can be reused on a larger vehicle, while the physical F450 is treated as the prototype implementation.

1.4 Applications

Application	How AERIS contributes
Flood reconnaissance	Scan exposed areas, identify visible people/vehicles/debris, and provide location-associated leads to responders.
Earthquake reconnaissance	Inspect blocked roads, damaged open areas, building-adjacent zones and visible survivor locations without initially sending personnel into unknown areas.
Landslide/terrain inspection	Provide aerial situational awareness over unstable or inaccessible ground.
Cyclone/post-storm assessment	Inspect affected sectors and identify visible people, vehicles or debris requiring follow-up.
Urban emergency support	Search a controlled area where roads or structures make immediate ground reconnaissance difficult.
Remote-area reconnaissance	Cover a pre-defined search area where ground access is slow or fragmented.
Emergency logistics demonstration	Travel to a verified target coordinate and release a small demonstration package after thrust, payload and flight validation.

1.5 Example mission scenarios

Scenario A - Search and report. The operator draws Search Area A. AERIS takes off, follows the generated coverage route, receives camera frames, detects a person, confirms the track, calculates an approximate target coordinate, assigns a priority, and sends an event to the ground station. The aircraft can continue the search while the operator reviews the event.

Scenario B - Point-to-point response. The operator supplies a target coordinate. AERIS navigates there using ArduPilot, maintains its configured response altitude, pauses for operator authorization, and releases only the small validated demonstration payload. It then returns home.

Scenario C - Search, respond, resume. AERIS searches Sector A, generates an incident, waits for human verification, performs a response mission, returns to the original mission context, and resumes from a predefined or regenerated waypoint rather than inventing a new route.

2. Prototype Scope and Capability Boundary

The supplied final build plan is intentionally conservative: the physical system is one small quadcopter, not a claimed 5-kg aircraft. The physical prototype demonstrates the software/mission architecture and limited flight functions; the 5-kg emergency logistics concept is represented in the digital twin only [D5].

Capability	Physical prototype	Digital twin / future
Autonomous takeoff/landing	Yes, after manual flight gate	Yes
GPS waypoint mission	Yes	Yes
Lawnmower/grid search	Yes	Yes
Person detection	Yes	Yes
Vehicle detection	Yes if trained/validated	Yes
Debris detection	Yes if trained/validated	Yes
Fire/smoke detection	Not a frozen primary claim; requires dedicated model validation	Can be simulated
Geo-tagging	Approximate target location with ground projection; fallback to aircraft position	Full simulated pipeline
Priority/decision layer	Rule-based	Rule-based / extensible
Forward obstacle sensing	Yes	Yes / simulated
360-degree obstacle avoidance	No	Possible future research
Payload	Small human-authorized demo only	5-kg target model only
Return-to-home / failsafe	Yes	Yes
Ground station	Yes	Yes
Live video	Local Wi-Fi demonstration	Simulated/replayed

NON-CLAIMS: The prototype is not a production search-and-rescue aircraft, not an all-weather platform, not a 5-kg heavy-lift aircraft, and not an autonomous life-critical decision maker [D5].

3. System Requirements and Success Criteria

3.1 Functional requirements

ID	Requirement	Implementation
FR-01	Mission input	Operator can define a target coordinate or a polygonal search area.
FR-02	Flight autonomy	ArduPilot executes takeoff/waypoint/landing behavior under configured limits.
FR-03	Manual override	Pilot can immediately override through the ELRS link.
FR-04	Telemetry	Ground station receives position, altitude, speed, battery, mode and mission status.
FR-05	Vision	Camera supplies frames to the Pi; detector runs locally on the Hailo NPU.
FR-06	Detection	Detector outputs class, confidence and bounding box.
FR-07	Tracking	Repeated frames are associated to tracks to prevent alert storms.
FR-08	Geo-tag	Each confirmed event receives timestamp and approximate target position.
FR-09	Priority	Event receives normal/high/critical rule-based priority.
FR-10	Alert	Ground dashboard displays the event and location.
FR-11	Obstacle	TFmini distance is filtered and used for warning/hold behavior.
FR-12	Response	Operator can convert a verified incident into a response waypoint.
FR-13	Payload	Release requires explicit human authorization and safe-state checks.
FR-14	Logging	Flight and AI events are stored locally with synchronized timestamps.

ID	Requirement	Implementation
FR-15	SITL	The mission manager/event schema can run against ArduPilot SITL.

3.2 Non-functional requirements

ID	Area	Acceptance statement
NFR-01	Safety	Flight controller must remain capable of stable flight without the Pi or AI process.
NFR-02	Determinism	Mission state transitions must be explicit and logged.
NFR-03	Recoverability	Software processes must restart without silently changing mission state.
NFR-04	Traceability	Every incident must have a unique event_id and mission_id.
NFR-05	Reproducibility	Model file, software versions, configuration and parameters are frozen in Git.
NFR-06	Thermal	Pi temperature and throttling state are logged during AI flight tests.
NFR-07	Electrical	All regulated rails are verified under worst expected load before flight.

3.3 Acceptance philosophy

A feature is considered implemented only when it passes a repeatable test. A feature that works once but has no log, no test procedure, or no defined failure behavior is treated as experimental, not accepted. This rule prevents the documentation from over-claiming capabilities simply because a component is installed.

4. Final Hardware Bill of Materials

The BOM below is the refined prototype BOM. Planning prices are based on the supplied final build plan and are intended for budgeting rather than a guaranteed retail quotation. Current retail prices vary by seller, GST/shipping, stock and exact board revision. The final purchase order should record the exact part number and supplier.

#	Item	Specification	Qty	Role	Planning price
1	F450 frame kit	450 mm X-frame; include 4x A2212 1000KV, 4x 30A ESC and 1045 prop set	1 kit	Airframe + propulsion	₹4,149
2	Pixhawk 2.4.8 + GPS/power	FMUv2-class board; M8N-class GPS/compass; power module	1 set	Flight control	₹12,000 budget
3	Raspberry Pi 5 4GB	4GB RAM; USB-C input; CSI camera; UART/USB	1	Companion computer	₹11,000 budget
4	Raspberry Pi AI HAT+ 13 TOPS	Hailo-8L, 13 TOPS INT8	1	AI accelerator	₹6,854
5	Camera Module 3 standard	12MP IMX708; standard 75-degree diagonal FoV	1	Primary perception	₹2,389
6	TFmini Plus	0.1-12 m class; UART/I2C; forward range	1	Obstacle sensing	₹3,500 budget
7	3S 11.1V 5200mAh LiPo	40-80C planning specification, XT60	1	Main battery	₹2,561
8	RadioMaster Pocket ELRS	2.4 GHz ELRS transmitter	1	Manual override	₹6,899
9	RadioMaster RP2 ELRS receiver	2.4 GHz CRSF receiver	1	Aircraft-side RC	₹1,200 budget
10	SiK telemetry pair	433/915 MHz legal/available for site; MAVLink	1 pair	GCS telemetry	₹3,940 budget
11	Raspberry Pi Active Cooler	Pi 5 compatible	1	Thermal management	₹600 budget
12	128GB high-endurance microSD	A2/U3 class or equivalent	1	OS/model/log storage	₹600 budget
13	3S balance charger	Balance charge, storage charge, correct chemistry	1	Battery charging	₹1,500

#	Item	Specification	Qty	Role	Planning price
14	5.1V >=5A buck converter	Dedicated Pi rail from 3S battery/PDB; regulated output	1	Pi power	₹1,200 budget
15	5V 3A servo BEC	Dedicated servo rail	1	Payload servo power	₹400 budget
16	CP2102N/FTDI 3.3V USB-UART	True 3.3V logic; USB side to Pi	1	Pi-Pixhawk MAVLink	₹300 budget
17	Micro servo + mechanical latch	SG90/MG90S-class or stronger; lightweight latch	1	Payload release	₹600 budget
18	Buzzer + status LED set	Pixhawk-compatible audible/visual indicators	1 set	State indication	₹250 budget
19	Sensor/mount hardware	3D printed/bolt-on mounts; vibration isolation	1 set	Mechanical integration	₹1,000 budget
20	Wiring/integration kit	XT60, wire, connectors, heat-shrink, fuse/protection, cable ties	1 lot	Electrical integration	₹1,200 budget
21	Propeller spares	2-4 extra sets + fasteners	1 lot	Field reliability	₹1,000 budget
22	Transmitter batteries	2x 18650 Li-ion cells	1 set	Pocket power	₹400 budget

PLANNING TOTAL: Approximately ₹63,542 for the refined engineering BOM. This is a planning figure; retail procurement can be higher. A laptop/ground-station computer is assumed already available and is not included in this subtotal.

PROCUREMENT RULE: The exact Pixhawk board, exact ESC model, exact BEC/buck model, exact receiver and exact telemetry radio must be recorded with manufacturer and revision. Do not substitute an electrically different part merely because the name is similar.

4.1 Optional / future items not required for the first flight

Item	Why it is optional	Reason for exclusion from frozen prototype
Second camera	Could add rear/nadir redundancy	Adds mass, power and software complexity; first prototype uses one calibrated camera.
Optical-flow sensor	Useful when GPS is unreliable	Not required for the primary outdoor GPS mission.
360-degree lidar	Would improve obstacle perception	Incompatible with the current budget and weight/compute targets.
Thermal camera	Useful for night/search applications	Not part of the frozen first prototype.
AI HAT+ 26 TOPS	More compute headroom	13 TOPS is sufficient for the selected baseline object detector.

5. Hardware Technical Design

5.1 Airframe and propulsion

The airframe is an F450-class X quadcopter. The vehicle uses four A2212 1000KV brushless motors, four 30A ESCs, and two CW plus two CCW 1045 propellers. Attitude control is achieved by differential motor speed; there is no mechanical drivetrain. The supplied technical documents explicitly describe the aircraft as a fixed-rotor quadcopter with no chain or sprocket drivetrain [D3].

Corrected Quad-X Motor Layout

Default ArduPilot Quad-X mapping for this specification

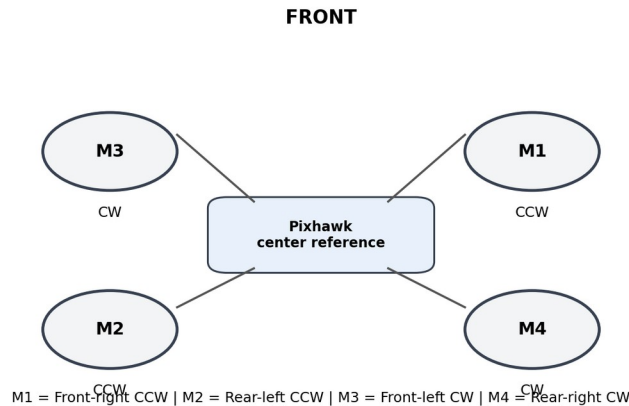


Figure 2. Corrected standard Quad-X motor numbering and rotation. Verify the purchased motor/prop pair before flight.

5.2 Flight controller

Pixhawk is the safety-critical controller. It performs IMU processing, attitude stabilization, GPS navigation, waypoint execution, flight-mode control, battery/GPS/communication failsafes and actuator output. The companion computer is not allowed to be a hard dependency for basic stable flight.

5.3 GPS and compass

Use the bundled M8N-class GPS/compass module with the highest practical physical separation from high-current ESC/motor wiring. Place the GPS/compass above the central electronics on a small mast and route the cable away from the PDB and motor leads where practical. Perform compass calibration only after final wiring and mechanical placement are complete.

5.4 Companion computer

The Raspberry Pi 5 is the mission and AI host. It receives images from the CSI camera, range samples from TFmini, MAVLink telemetry from Pixhawk, and operator commands via the local network/dashboard. It owns high-level decisions but must not replace the flight controller safety loop.

5.5 AI accelerator

The Raspberry Pi AI HAT+ provides the Hailo-8L NPU and is available in a 13 TOPS variant. Raspberry Pi states that AI HAT+ hardware-accelerates supported AI workloads locally on Raspberry Pi 5 and integrates with the camera/vision stack [W1][W2].

5.6 Camera

The final prototype uses the standard Camera Module 3 rather than the wide model. Raspberry Pi specifies a 12MP IMX708 sensor with a 75-degree diagonal field of view for the standard variant; the Wide variant is 120 degrees [W3]. The standard lens is preferred for the first prototype because excessive width can reduce the pixel footprint of a small person at altitude. The camera should be mounted near-nadir or with a small, repeatable forward tilt for the search mission.

5.7 Obstacle sensor

TFmini Plus is used as a lightweight forward range sensor. Its role is deliberately limited to forward obstacle awareness. The prototype first implements warning/slow/hold behavior. Automatic rerouting is not considered a guaranteed capability until the hold/stop behavior is repeatedly demonstrated in flight.

5.8 RC override

The RadioMaster Pocket and RP2 ELRS receiver form an independent manual override path. ELRS/CRSF is a UART-based digital protocol, so the receiver is connected to a true Pixhawk UART rather than pretending it is an RC PWM input. The receiver remains live during every flight.

5.9 Telemetry

SiK telemetry carries MAVLink between Pixhawk and the ground station. It is intentionally not used as the live-video transport. The local Wi-Fi link is used for video and AI event dashboard traffic. This separation prevents the low-bandwidth telemetry link from being overloaded by video.

5.10 Payload actuator

The servo latch is driven from an AUX output through the Pixhawk gripper function. ArduPilot supports a servo gripper on an AUX output with SERVOx_FUNCTION=28 and control through MAV_CMD_DO_GRIPPER [W7]. The servo receives power from a dedicated 5V/3A BEC. The physical release remains human-authorized and is blocked unless the mission state, position and safety checks permit it.

6. Power, Weight, Thrust and Energy Engineering

6.1 Battery energy

Nominal battery energy = $V_{\text{nominal}} \times \text{Ah} = 11.1 \text{ V} \times 5.2 \text{ Ah} = 57.72 \text{ Wh}$

80% usable energy = $57.72 \times 0.80 = 46.18 \text{ Wh}$

75% usable energy = $57.72 \times 0.75 = 43.29 \text{ Wh}$

Average electrical power	Time at 80% usable	Time at 75% usable
250 W	11.08 min	10.39 min
275 W	10.07 min	9.44 min
300 W	9.24 min	8.66 min
325 W	8.53 min	7.99 min
350 W	7.92 min	7.42 min

INTERPRETATION: The physical plan uses an 8-10 minute planning estimate, but exact flight time must be measured on the purchased aircraft. Camera, Pi, AI HAT, payload, wind, battery age and the actual hover power all change the result.

6.2 Thrust-to-weight check

Available thrust depends strongly on the purchased A2212 motor, propeller, battery voltage and test conditions. The planning range below uses a conservative 700 g to 800 g per motor to show feasibility rather than promise a specific result. Four motors therefore represent approximately 2.8 kg to 3.2 kg of total static thrust before losses and without accounting for maneuver reserve.

Scenario	Assumed total mass	Total thrust at 700 g/motor	T/W	Total thrust at 800 g/motor	T/W
Prototype, no payload	1.45 kg	2.80 kg	1.93	3.20 kg	2.21
Prototype + 0.25 kg demo load	1.70 kg	2.80 kg	1.65	3.20 kg	1.88
Prototype + 0.40 kg demo load	1.85 kg	2.80 kg	1.51	3.20 kg	1.73

PAYLOAD GATE: The payload is not approved by calculation alone. We require a measured static thrust test, measured all-up mass, hover-current test and flight test. The release mechanism remains optional until those measurements pass.

6.3 Electrical current envelope

Example short-duration motor current envelope (planning): $12 \text{ A} \times 4 \text{ motors} = 48 \text{ A}$

At 12.6 V full battery: $12.6 \text{ V} \times 48 \text{ A} = 604.8 \text{ W}$ electrical input before ESC/motor efficiency losses

This is not a claim that the aircraft will draw 48 A continuously. It is a sizing check showing why the PDB, XT60, wiring and ESC paths need adequate current margin.

6.4 Pi power budget

Pi design rail = 5.1 V, 5 A continuous-capable
 Maximum rail power capability = 5.1 V x 5 A = 25.5 W
 At 90% buck efficiency from 11.1 V nominal battery:
 Battery current at 25.5 W output = 25.5 / (11.1 x 0.90) = 2.55 A

Raspberry Pi documentation recommends 5V at 5A for Raspberry Pi 5; a 3A supply constrains available peripheral current [W4]. The dedicated buck therefore avoids the ambiguous “generic BEC” problem in the earlier document.

6.5 Search geometry

The standard Camera Module 3 has a 75-degree diagonal FoV. For planning, a horizontal FoV around 66 degrees is used. The approximate ground swath width is $W = 2 h \tan(\text{FOV}_h/2)$. With 20% lane overlap, lane spacing is $S = W \times (1 - 0.20)$.

At $h = 8$ m, $\text{FOV}_h = 66$ deg:
 $W \sim 2 \times 8 \times \tan(33 \text{ deg}) \sim 10.39$ m
 Lane spacing $\sim 10.39 \times 0.80 = 8.31$ m
 At $h = 10$ m:
 $W \sim 12.98$ m
 Lane spacing ~ 10.38 m

These are geometric coverage estimates, not detection guarantees. The real minimum usable altitude must be selected from actual person-pixel size, detector performance, safe flight rules, camera exposure and environmental conditions.

6.6 Obstacle stopping-distance calculation

The first TFmini behavior uses a conservative speed of 3 m/s. A planning safety distance is calculated from reaction latency, deceleration and a margin.

Reaction distance = $v \times t = 3.0 \times 0.20 = 0.60$ m
 Stopping distance = $v^2 / (2a) = 9 / (2 \times 1.0) = 4.50$ m
 Safety margin = 1.50 m
 Total = $0.60 + 4.50 + 1.50 = 6.60$ m

The initial firmware/software configuration therefore uses a warning zone around 8 m and a hold/stop action around 6.5 m, subject to outdoor flight test. These numbers are engineering starting values, not guaranteed stopping performance. They must be re-tuned from measured braking response.

6.7 Geo-location error budget

Target geo-location is the sum of GPS error, attitude error, altitude error, camera calibration error, object-centroid error and local ground-height error. If these errors were independent, a root-sum-square estimate can show the scale, but it should never be presented as a guarantee.

Illustrative open-sky target: GPS 2.0 m, camera/centroid 1.0 m, attitude projection 0.5 m, altitude/ground 0.7 m
 $\text{RSS} \sim \sqrt{2.0^2 + 1.0^2 + 0.5^2 + 0.7^2} \sim 2.4$ m

MEASUREMENT REQUIREMENT: The prototype acceptance target should be stated as a measured 95th-percentile or similar field error, not the illustrative calculation above. Use surveyed markers at known coordinates during calibration.

7. Final Software Technical Stack

Layer	Technology	Where	Job
Flight firmware	ArduPilot Copter	Pixhawk	Stabilization, GPS navigation, modes, mission execution, failsafe, actuator control
Protocol	MAVLink	Pixhawk <-> Pi <-> GCS	Telemetry, mission commands, status, gripper commands
GCS	Mission Planner	Ground laptop	Parameter setup, mission planning, logs, telemetry
Backup GCS	QGroundControl	Ground laptop	Secondary mission/telemetry interface
Companion OS	Raspberry Pi OS 64-bit	Pi 5	System runtime

Layer	Technology	Where	Job
Language	Python 3	Pi 5	Mission logic, event manager, geo-tagging, dashboard services
Camera	Picamera2 / rpicam	Pi 5	CSI camera capture and frame control
CV	OpenCV	Pi 5	Resize, color, drawing, calibration, image transforms
AI runtime	HailoRT + Hailo software	AI HAT+	NPU inference
Detector	Custom YOLO11n -> Hailo HEF	AI HAT+	Person/vehicle/debris detection
Tracker	ByteTrack	Pi CPU	Track IDs and temporal association
Coverage planning	Polygon clipping + lawnmower sweep	Pi or GCS	Generate systematic search waypoints
Obstacle logic	Median filter + threshold state machine	Pi	TFmini warning/hold behavior
Geo-tagging	Camera model + pose transform + ground-plane intersection	Pi	Approximate target coordinate
Priority	Rule-based weighted score	Pi	Normal/high/critical triage
Data store	SQLite + JSONL	Pi	Mission/event logging
Dashboard	FastAPI + WebSocket + Leaflet/OpenStreetMap	Ground laptop	Events, map, telemetry subset, video UI
Video	Local Wi-Fi; MJPEG/RTP/WebRTC implementation selected at build	Pi -> laptop	Live perception feed
Simulation	Gazebo + ArduPilot SITL	Dev PC	Digital-twin testing
Version control	Git + GitHub	Dev PC	Code/config/version traceability
CAD	FreeCAD	Dev PC	Mounts, payload box, physical geometry
Training	PyTorch + Ultralytics	Development PC	Custom detector training and evaluation

7.1 Current Hailo deployment rule

Ultralytics currently documents Hailo export for YOLOv8/YOLO11 detection on Hailo-8/Hailo-8L. The model is compiled to a hardware-specific HEF on a Linux x86_64 workstation, while inference occurs on the Raspberry Pi through HailoRT. The input shape is fixed at export time; custom models should be calibrated with representative production-domain images [W10].

7.2 Software version freeze

Artifact	Freeze rule
ArduPilot firmware	Record exact stable build and firmware hash/version used in the flight controller.
Raspberry Pi OS	Record image release and SHA/manifest; current Raspberry Pi AI documentation targets 64-bit Raspberry Pi OS Trixie for AI on Pi 5 [W5].
Python	Record exact major/minor and pip lock/requirements.txt.
HailoRT	Record exact package version and accelerator identification output.
Ultralytics	Record exact package version and commit/package hash.
YOLO HEF	Store HEF plus metadata in Git LFS or release artifact store; record DFC version.
Mission parameters	Store complete ArduPilot parameter file as an immutable release artifact.
Dashboard	Pin dependencies and store a reproducible requirements lock.

8. AI Model, Data, Mathematics and Algorithms

8.1 Detection model

The first physical detector is a custom-trained YOLO11n object detector with three frozen primary classes: person, vehicle and debris. A fire/smoke class is not claimed in the first physical system unless a dedicated dataset and evaluation prove it sufficiently. This keeps the prototype aligned with what the trained model can actually recognize.

YOLO-style detection formulates object localization and classification in a unified neural network. The original YOLO paper introduced the approach as a unified real-time detection system [R1]. The implementation here uses

the current YOLO11n family because current Ultralytics documentation validates YOLO11 detection export to Hailo-8L/8 [W10].

8.2 Training dataset strategy

- Collect or source license-compatible imagery that resembles the prototype: aerial/downward or oblique views, small people, partially occluded people, vehicles, rubble/debris and mixed backgrounds.
- Create train/validation/test splits by scene or location where possible so nearly identical frames do not leak across the splits.
- Apply realistic augmentation: scale, crop, brightness/contrast, blur, mild rotation and sensor-noise variations. Do not over-augment beyond the camera physics.
- Calibrate the Hailo model using representative production-like images rather than generic images. Ultralytics recommends a large, diverse calibration set for Hailo export; its current guide recommends at least 1,024 calibration images for best accuracy [W10].
- Freeze dataset version, licenses, class names and label format before training a release model.

8.3 Baseline model configuration

Parameter	Initial value	Why
Model	YOLO11n detector	Small edge baseline compatible with Hailo-8L
Input	640x640	Balanced edge accuracy/compute; fixed at Hailo export
Classes	person, vehicle, debris	Frozen physical claims
Detector confidence	0.35 baseline	Retains candidate detections for tracking
Event confidence	0.60 baseline	Higher bar for ground alert
NMS IoU	0.45 baseline	Suppress duplicate boxes
Inference target	15-20 FPS if measured feasible	Leaves CPU for mission logic
Tracker	ByteTrack	Temporal association and track IDs
Event confirmation	3 of last 5 frames	Reduces single-frame false alarms
Cooldown	10 s + spatial cell check	Prevents repeated incident spam

TUNING RULE: The values above are starting configurations. The release threshold is the value measured on the actual model/scene test set, not a number chosen only because it looks reasonable.

8.4 ByteTrack role

ByteTrack associates detections across frames so the system can distinguish a persistent object from a one-frame false positive. The original ByteTrack paper reports an association strategy that also uses lower-score detections to recover fragmented trajectories [R2]. In AERIS, the main benefit is operational: stable track IDs make event confirmation and de-duplication possible.

8.5 Detection/event pipeline mathematics

For each detection $d_i = (\text{class}, \text{confidence}, \text{bbox})$, keep a candidate if confidence ≥ 0.35 .
 Use the tracker to associate boxes to track k .
 Confirm a track if detections are present in ≥ 3 of the last 5 inference frames.
 Generate one event per confirmed track unless cooldown/spatial suppression permits another event.

8.6 Priority scoring

Use a transparent weighted rule instead of a black-box learned triage model. Let c be detector confidence in $[0,1]$, t be persistence score in $[0,1]$, s be class severity in $[0,1]$, and m be mission-context severity in $[0,1]$.

$$\text{priority_score} = 100 * (0.55*c + 0.20*t + 0.15*s + 0.10*m)$$

Class	Severity s	Reason
person	1.00	Direct life-safety relevance for the demo
vehicle	0.70	Potential incident/supporting evidence
debris	0.50	Hazard/context indicator

Score	Priority	Ground action
≥ 80	CRITICAL	Immediate visual/audible attention; response still requires human authorization.

Score	Priority	Ground action
60-79	HIGH	Operator review / potential response.
40-59	NORMAL	Record and continue search.
< 40	LOG ONLY	Store but do not escalate.

8.7 Coverage path planning

AERIS uses a lawnmower/grid coverage pattern. Coverage path planning is the task of generating a path that covers an area under vehicle constraints; survey literature identifies path length, coverage, obstacles, and energy as key factors [R3][R4]. The first implementation is intentionally deterministic: polygon -> parallel sweep lines -> lane clipping -> alternating direction -> entry/exit waypoints.

Given search polygon P, altitude h and horizontal FoV theta:

$$W = 2 * h * \tan(\theta / 2)$$

$$S = W * (1 - \text{overlap})$$

Generate parallel lanes spaced S, clip each lane to P, alternate direction, add turn waypoints.

A 20% overlap is the initial planning value. The mission speed is 2.5-3.0 m/s for a controlled demonstration. The route generator should reduce lane spacing if actual detection quality requires it, even if the resulting path is longer.

8.8 Geo-tagging mathematics

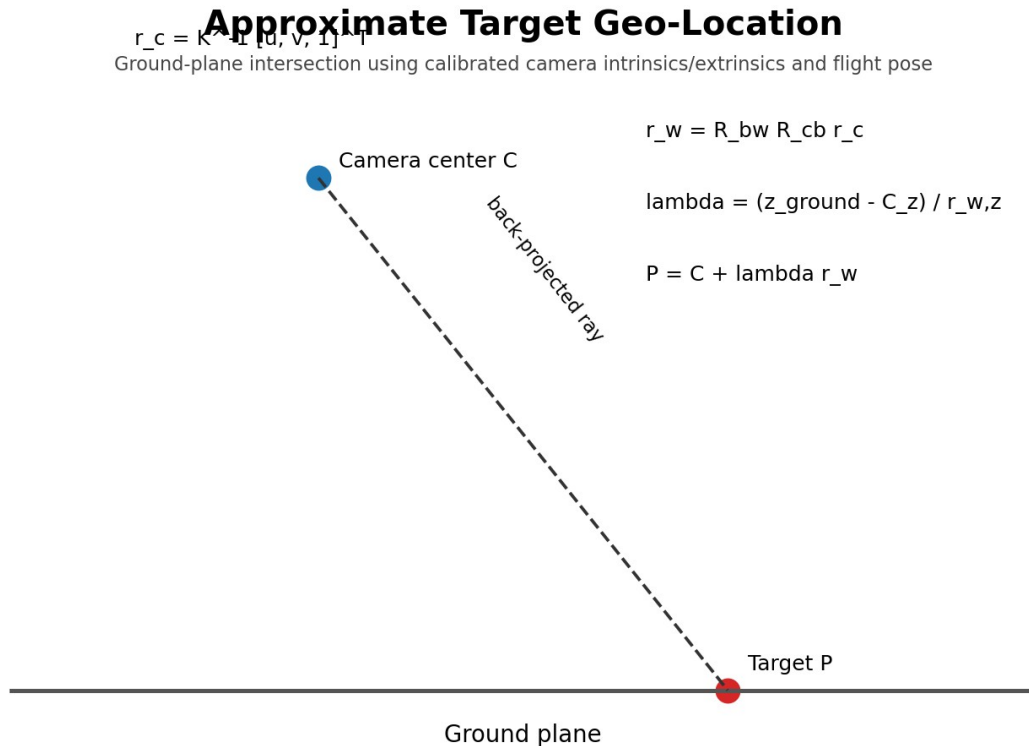


Figure 7. Approximate target geo-location geometry.

Camera ray: $r_c = K^{-1} [u, v, 1]^T$

World ray: $r_w = R_{bw} * R_{cb} * r_c$

Ground intersection: $\lambda = (z_{\text{ground}} - C_z) / r_{w,z}$

Target point: $P = C + \lambda * r_w$

Here K is the camera intrinsic matrix, R_{cb} is the fixed camera-to-body rotation, R_{bw} is the vehicle/body-to-world rotation derived from flight attitude, C is the camera position in the world frame, and z_{ground} is the local ground-plane height. The prototype uses this to estimate a target coordinate. When the geometric solution is invalid or poorly conditioned (for example, the ray points toward the horizon), the system records the aircraft GPS position as a fallback and explicitly labels the location as approximate.

8.9 Obstacle filter

Let d_k be the TFmini distance samples.
 Filtered distance = median($d_{(k-2)}$, ..., $d_{(k+2)}$)
 Reject samples outside the physical sensor range or obvious jumps.
 State: NORMAL if $d > 8.0\text{m}$
 WARNING if $6.5\text{m} < d \leq 8.0\text{m}$
 HOLD if $d \leq 6.5\text{m}$

The exact thresholds are tuned after observing real stopping distance and false detections. The first safe behavior is hold/pause, not a complex autonomous reroute.

8.10 Quadrotor dynamics model

The controller itself is ArduPilot, not a custom controller developed inside this prototype. The mathematics below explains the physical system and is useful for the digital twin and validation. A simplified rigid-body model is:

$$\begin{aligned} m \cdot \dot{v} &= m \cdot g + R \cdot f_b \\ R \cdot \dot{\omega} &= R \cdot \text{skew}(\Omega) \cdot \omega \\ J \cdot \dot{\Omega} + \Omega \times (J \cdot \Omega) &= \tau \end{aligned}$$

m is vehicle mass, v is inertial velocity, R is body-to-world rotation, f_b is total thrust in body coordinates, J is inertia, Ω is body angular velocity, and τ is the control moment. Each motor contributes thrust and reaction torque; the net roll/pitch/yaw moments arise from differential motor force and drag torque. These equations are used to reason about thrust margin and to set realistic SITL/digital-twin parameters.

9. Full Hardware-Software Architecture

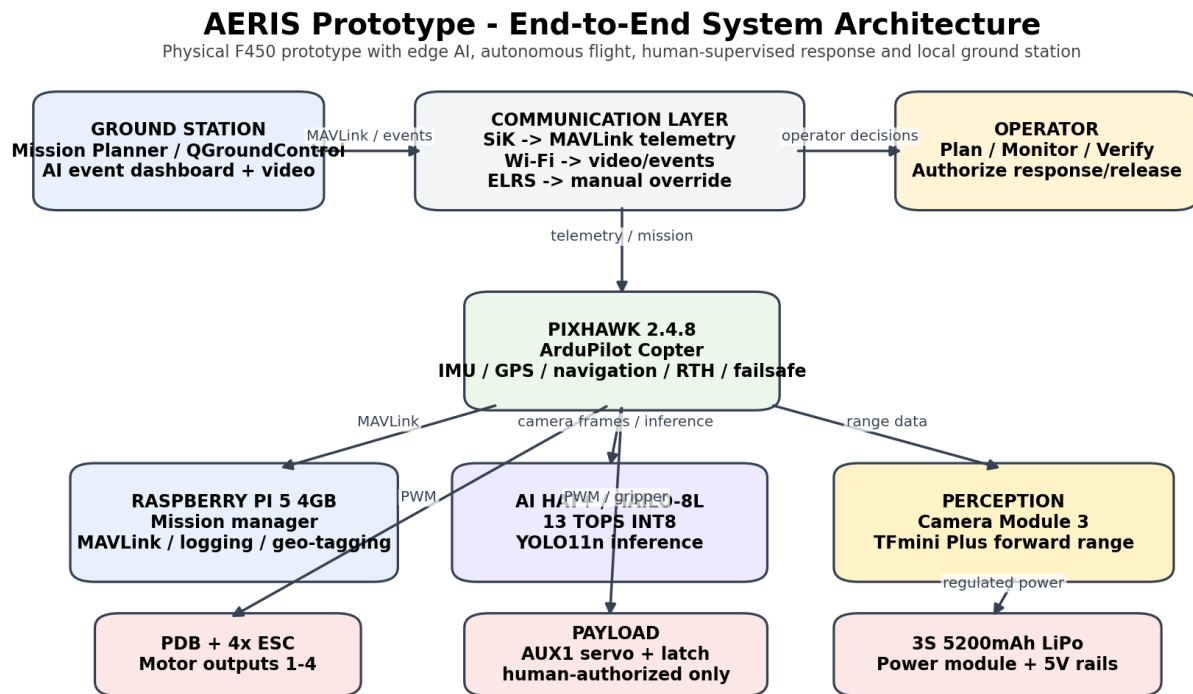


Figure 1 (repeated for reference). Final end-to-end architecture.

9.1 Hardware responsibility split

Layer	Hardware	Responsibility	Failure behavior
Energy	LiPo + PDB + power module + regulators	Provide independent rails to flight, Pi and payload	Undervoltage is detected; battery failsafe acts through Pixhawk.
Propulsion	Motors + ESCs + props	Generate thrust and control moments	Lost actuator is a flight fault; no AI process can safely repair it.
Flight	Pixhawk + IMU + barometer + GPS	Stabilization, navigation, RTH, failsafe	Continues basic flight control without Pi.

Layer	Hardware	Responsibility	Failure behavior
Companion	Pi 5	Mission orchestration, event processing, logs	Watchdog restarts process; aircraft remains under Pixhawk control.
AI	AI HAT+ Hailo-8L	Low-latency object inference	AI can stop; flight remains safe.
Vision	Camera Module 3	Image acquisition	AI event pipeline reports camera fault.
Range	TFmini Plus	Forward distance	Fails safe to conservative hold behavior when sensor invalid.
Communication	SiK + Wi-Fi + ELRS	Telemetry, video/events, pilot override	ELRS remains the independent manual channel.
Payload	Servo + latch	Physical demo release	Release is blocked unless authorized and preconditions pass.

9.2 Data ownership rule

Each data item has one authoritative owner. Flight attitude, mode, GPS position and battery state come from Pixhawk. Raw camera frames come from the camera/Pi stack. AI detections come from the Hailo inference process. Mission state is owned by the Pi mission manager but must never command the flight controller outside an allowed command set. Ground-station authorization is authoritative for payload release.

9.3 End-to-end data flow

Camera -> Pi capture -> Hailo YOLO -> detections -> ByteTrack -> confirmed track
-> geo-tag -> priority -> incident event -> Wi-Fi dashboard + JSONL/SQLite

Pixhawk -> MAVLink -> Pi telemetry cache -> mission/event association -> dashboard
Pixhawk -> SiK -> Mission Planner/QGC
ELRS TX -> RP2 -> Pixhawk -> manual override
TFmini -> Pi -> obstacle state -> mission manager -> pause/hold
Operator -> dashboard -> response authorization -> Pi -> MAVLink -> Pixhawk -> AUX1 servo

10. Electrical and Communication Interface Specification

10.1 Pixhawk port plan

Interface	Use	Connection	Notes
POWER	Power module	Power module to Pixhawk POWER port	Carries regulated 5V plus current/voltage sense; verify exact board connector pinout.
MAIN OUT 1	Motor 1	ESC1 signal + GND	ArduPilot Motor 1.
MAIN OUT 2	Motor 2	ESC2 signal + GND	ArduPilot Motor 2.
MAIN OUT 3	Motor 3	ESC3 signal + GND	ArduPilot Motor 3.
MAIN OUT 4	Motor 4	ESC4 signal + GND	ArduPilot Motor 4.
TELEM1 / SERIAL1	SiK telemetry	Pixhawk UART <-> air SiK radio	MAVLink; hardware flow control if radio supports it.
TELEM2 / SERIAL2	Companion link	Pixhawk UART <-> 3.3V USB-UART <-> Pi USB	MAVLink; use dedicated adapter.
GPS	M8N GPS	GPS UART + compass harness	GPS position; compass may use I2C within bundled harness.
SERIAL4	ELRS CRSF	Pixhawk UART <-> RP2	Full-duplex CRSF; true UART.
AUX OUT 1 / SERVO9	Payload servo	Servo signal + external 5V/3A BEC + GND	SERVO9_FUNCTION=28 for ArduPilot gripper.
BUZZER	Buzzer	Pixhawk buzzer port	State/failsafe audible output.
SAFETY SWITCH	External switch	Pixhawk safety switch port	Follow exact board/harness instructions.

BOARD-SPECIFIC CAUTION: Pixhawk 2.4.8 products are not all identical. The pin tables below describe the standard FMUv2-style logical connection; verify the exact purchased board manual/silkscreen before applying power. Never rely on a photo of another Pixhawk clone as a wiring authority.

10.2 Raspberry Pi interfaces

Pi interface	Pin	Signal / endpoint	Purpose
5V power	Pin 4	5.1V buck output	Primary Pi power
GND	Pin 6	Common ground	Power/reference

Pi interface	Pin	Signal / endpoint	Purpose
UART TXD	GPIO14, physical pin 8	Pi TX -> TFmini RX	TFmini command/configuration if required
UART RXD	GPIO15, physical pin 10	TFmini TX -> Pi RX	TFmini range data
USB	USB port	3.3V-logic USB-UART adapter	Pixhawk MAVLink link
CSI camera	Camera connector	Camera Module 3 ribbon	Vision input
PCIe/HAT	AI HAT+ stack interface	Hailo-8L	AI inference
Wi-Fi	Onboard radio	Local network	Video + event dashboard

11. Pin-by-Pin Wiring and Connection Tables

WIRING PRINCIPLE: Make all signal connections with the battery disconnected. Use continuity and polarity checks before energizing a new rail. For UARTs, remember that TX connects to RX and RX connects to TX.

11.1 LiPo to power tree

From	Pin/wire	To	Pin/wire	Signal
3S LiPo XT60 +	Positive	PDB +	Battery +	Main positive bus
3S LiPo XT60 -	Negative	PDB -	Battery -	Main ground bus
PDB battery bus	Positive	Power module IN+	IN+	Voltage/current monitor input
PDB battery bus	Negative	Power module IN-	IN-	Monitor ground
PDB battery bus	Positive	5.1V buck IN+	IN+	Pi regulator input
PDB battery bus	Negative	5.1V buck IN-	IN-	Pi regulator ground
PDB battery bus	Positive	5V servo BEC IN+	IN+	Servo regulator input
PDB battery bus	Negative	5V servo BEC IN-	IN-	Servo regulator ground

11.2 Pixhawk POWER port

Pixhawk POWER pin	Electrical role	Connect to
Pin 1	+5V	Power module regulated +5V
Pin 2	+5V	Power module regulated +5V
Pin 3	CURRENT analog	Power module current-sense output
Pin 4	VOLTAGE analog	Power module voltage-sense output
Pin 5	GND	Power module ground

VERIFY: Use the exact power-module harness shipped with the exact Pixhawk board. Do not cut/re-pin a connector unless the board manual confirms the same pinout.

11.3 Motor/ESC signal wiring

Pixhawk output	ESC	Motor position	Motor rotation	Prop
MAIN OUT 1	ESC1	Front-right	CCW	1045 CCW
MAIN OUT 2	ESC2	Rear-left	CCW	1045 CCW
MAIN OUT 3	ESC3	Front-left	CW	1045 CW
MAIN OUT 4	ESC4	Rear-right	CW	1045 CW

Each ESC signal lead connects to the assigned Pixhawk output signal and its signal ground. With the dedicated servo BEC architecture, do not parallel multiple ESC BEC outputs onto the servo rail. Follow the purchased ESC manual for its three motor phase wires and battery-side connections. Brushless motor direction is corrected by swapping two of the three motor phase wires or by a supported ESC direction setting, then re-tested without propellers.

11.4 GPS/compass

Signal	Pixhawk GPS-side pin	M8N side	Purpose
+5V	GPS pin 1	GPS VCC	Module power
TX	Pixhawk RX (GPS pin 3)	GPS TX	GPS data into Pixhawk
RX	Pixhawk TX (GPS pin 2)	GPS RX	GPS configuration/data back to module
GND	GPS pin 6	GPS GND	Reference
I2C SDA	I2C SDA pin	Compass SDA	Compass data if separate
I2C SCL	I2C SCL pin	Compass SCL	Compass clock if separate

If the purchased M8N module uses a combined harness, use the manufacturer-supplied harness and verify continuity rather than assuming the wire order from another M8N board.

11.5 TFmini Plus to Raspberry Pi UART

TFmini Plus	Wire / logical pin	Raspberry Pi	Purpose
Pin 1	+5V	Physical pin 4 (5V)	Sensor power
Pin 2	RXD / SDA	GPIO14 / physical pin 8 TXD	Pi -> sensor commands
Pin 3	TXD / SCL	GPIO15 / physical pin 10 RXD	Sensor -> Pi distance
Pin 4	GND	Physical pin 6 GND	Reference

LOGIC LEVEL: TFmini Plus uses a 5V supply and 3.3V logic. Do not feed the sensor UART directly from a 5V-TTL adapter. Keep the Pi GPIO at 3.3V logic.

11.6 Pi to Pixhawk MAVLink via 3.3V USB-UART

Pixhawk TELEM2	USB-UART adapter	Raspberry Pi	Signal
Pin 2 TX	RX	USB RX	Pixhawk -> Pi
Pin 3 RX	TX	USB TX	Pi -> Pixhawk
Pin 6 GND	GND	USB GND	Common reference
Pin 1 +5V	DO NOT USE for adapter power	USB power from Pi	Avoid accidental parallel power rails

Set the Pixhawk serial port used by TELEM2 to MAVLink2 at the selected baud. The Pi opens the corresponding `/dev/ttyUSB\*` device. The adapter must be a 3.3V-logic device.

11.7 SiK telemetry pair

Pixhawk TELEM1	SiK air radio	Direction
Pin 1 +5V	5V/VCC	Power
Pin 2 TX	Radio RX	Pixhawk -> radio
Pin 3 RX	Radio TX	Radio -> Pixhawk
Pin 4 CTS	CTS	Optional hardware flow control
Pin 5 RTS	RTS	Optional hardware flow control
Pin 6 GND	GND	Reference

If the selected SiK radio cable exposes no flow control, use only TX/RX/5V/GND. The exact cable/pinout supplied with the radio pair is authoritative.

11.8 ELRS receiver on SERIAL4

Pixhawk SERIAL4	RP2 ELRS receiver	Signal
Pin 1 +5V	5V	Receiver power
Pin 2 TX4	Receiver RX	CRSF telemetry/control to receiver
Pin 3 RX4	Receiver TX	CRSF receiver data to Pixhawk
Pin 6 GND	GND	Reference

PARAMETER: Configure SERIAL4 for the CRSF/ELRS protocol according to the current ArduPilot documentation and the exact firmware version used. Do not use an RC input pin as a substitute for the required full UART connection.

11.9 AUX1 servo / gripper

AUX1 servo rail	Connection	Electrical detail
Signal S	Servo signal wire	Pixhawk AUX1 / SERVO9 signal
+5V +	5V servo BEC output	Dedicated 5V/3A rail
GND -	Servo BEC ground	Common ground with Pixhawk signal ground

```
SERV09_FUNCTION = 28
GRIP_ENABLE = 1
GRIP_TYPE = 1 (servo)
GRIP_GRAB / GRIP_RELEASE / GRIP_NEUTRAL = measured safe PWM values
Release command = MAV_CMD_DO_GRIPPER (after operator authorization and safety gates)
```

11.10 Buzzer and status indication

Connect the buzzer and safety switch using the exact Pixhawk harness supplied with the flight controller. Mount the buzzer so it is audible, but use appropriate mechanical isolation and placement; do not place noisy vibration sources directly on the flight-controller IMU mount.

11.11 Full hardware I/O flow

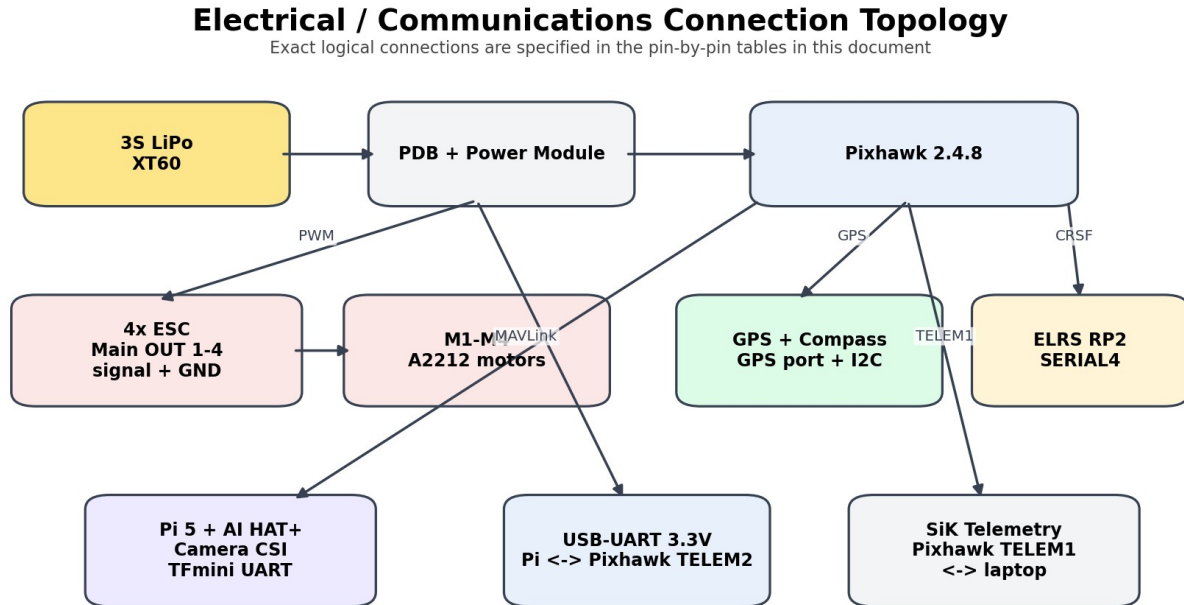


Figure 9. Connection topology.

12. Software Repository and Runtime Structure

```

aeris/
  firmware/
    ardupilot/
      parameters/
        prototype.param
  companion/
    vision/
      capture.py
      preprocess.py
      infer.py
      tracker.py
    sensors/
      tfmini.py
      telemetry.py
    geo/
      camera_model.py
      projection.py
    mission/
      state_machine.py
      coverage.py
      response.py
    events/
      event_manager.py
      priority.py
      schemas.py
    comms/
      mavlink.py
      websocket.py
      video.py
    logging/

```

```

db.py
jsonl.py
health/
  watchdog.py
main.py
dashboard/
  app.py
  static/
  templates/
models/
  yolol1n_aeris_hailo/
    model.hef
    metadata.yaml
    classes.txt
configs/
  mission.yaml
  ai.yaml
  sensor.yaml
  network.yaml
sim/
  sitl/
  gazebo/
  scenarios/
tests/
  unit/
  integration/
  hardware/
cad/
  frame_mounts/
  payload/
docs/
  pinmap.csv
  test_reports/

```

12.1 Processes and threads

Runtime component	Suggested cadence / trigger	Inputs	Outputs
Camera capture process	15-20 FPS	Camera	Latest-frame ring buffer
Hailo inference process	As fast as validated	Frames	Detections
Telemetry task	5-10 Hz	MAVLink	Vehicle state cache
TFmini task	Sensor rate / >=20 Hz sampled	UART range	Filtered distance
Mission manager	10 Hz	State + telemetry + obstacle + operator	Allowed flight commands
Event manager	On detection/track update	Detections + telemetry + pose	Incident records
GCS server	WebSocket event driven	Telemetry/events/video	Dashboard state
Logger	Buffered asynchronous	All important messages	SQLite + JSONL
Watchdog	1 Hz	Process/heartbeat/temperature	Restart/escalation

Use multiprocessing for camera/inference separation if needed; the Hailo pipeline can do the heavy neural work outside the Python CPU path. The objective is to keep the mission manager responsive even when an inference frame takes longer than expected.

13. Mission State Machine and Decision Logic

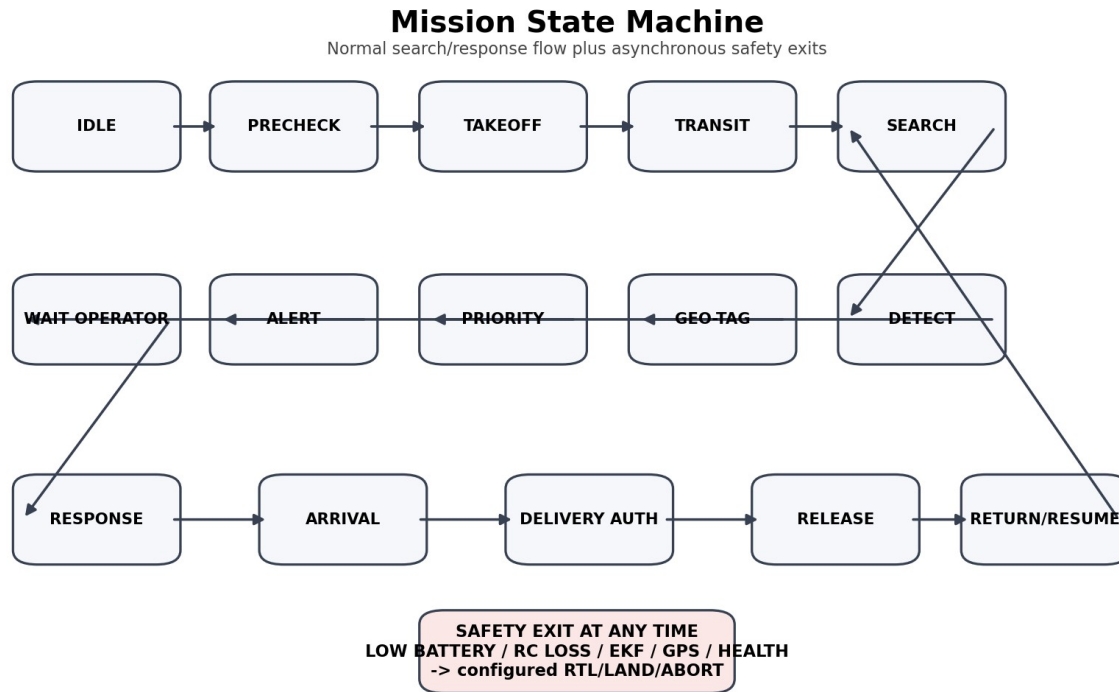


Figure 5. Final mission state machine.

State	Entry condition	Behavior
IDLE	No mission	Wait for operator mission and valid telemetry.
PRECHECK	Mission accepted	Battery, GPS, EKF, RC, compass, camera, TFmini and payload status checks.
TAKEOFF	Armed + authorized	Perform controlled takeoff to configured altitude.
TRANSIT	En route	Fly to search start / response waypoint.
SEARCH	Coverage route	Execute lawnmower waypoints; process perception.
DETECT_CONFIRM	Track detected	Apply 3-of-5 confirmation and confidence rules.
GEO_TAG	Confirmed incident	Calculate approximate target coordinate and error/status.
PRIORITIZE	Incident record	Compute rule-based priority score.
ALERT	Operator review	Send event to dashboard and log.
WAIT_OPERATOR	Verified/unverified	Ignore, mark, or authorize response.
RESPONSE	Authorized response	Fly to target under mission constraints.
ARRIVAL	At target	Check position, altitude, obstacle and safety state.
DELIVERY_AUTH	Release gate	Require explicit operator release and valid payload state.
RELEASE	Actuate	Command Pixhawk gripper; verify servo/release feedback if available.
RETURN/RESUME	After response	Return home or resume search from a known mission context.
ABORT	Any critical failure	Do not release; execute configured safety action.

13.1 Response authorization interlocks

- Operator authorization must be explicit, not inferred from a detection confidence score.
- Current vehicle mode must be one of the permitted response modes.
- Battery must be above the response reserve threshold and the return path must remain feasible.
- GPS/EKF health must pass the configured threshold.
- TFmini must not be in the obstacle HOLD state.

- Payload must be mechanically latched and servo position known.
- Only the mission manager may request the release command; random dashboard clicks must not directly pulse the servo.

13.2 Failsafe precedence

CRITICAL VEHICLE SAFETY > RC OVERRIDE > FLIGHT CONTROLLER FAILSAFE > MISSION ABORT > AI DECISION

The exact ArduPilot failsafe behavior is configured on the flight controller. The Pi can request RTL or abort, but it should not attempt to override a stronger flight-controller failsafe.

14. Full Implementation Roadmap

Phase	Work package	Primary work	Exit criterion
P0	Document and procurement freeze	Complete master spec, part numbers, parameter template, software version manifest and pin map.	Signed configuration baseline.
P1	Bench power	Verify battery, PDB, power module, Pi buck, servo BEC and all rails.	Measured voltage/current table.
P2	Frame/propulsion	Assemble frame, mount motors, ESCs, wiring, corrected Quad-X labels.	Dry motor test passed.
P3	Pixhawk manual flight	Install ArduPilot, calibrate, configure modes/failsafes, perform no-payload manual flight.	Stable manual/assist flight.
P4	Pi + AI HAT	Install Pi OS, HAT, cooler, camera, Hailo runtime and test image inference.	Stable bench inference.
P5	MAVLink	Connect Pi via USB-UART; read telemetry and send safe non-flight commands.	Live telemetry cache.
P6	Ground station	SiK link, Mission Planner, local Wi-Fi dashboard and video.	Operator view working.
P7	AI event layer	Custom model, tracking, de-duplication, event schema and logs.	Repeatable event generation.
P8	Geo-tag	Calibrate camera/pose transform and test surveyed targets.	Measured target error report.
P9	Coverage planner	Generate and upload lawnmower mission from polygon.	Search mission repeatability.
P10	Obstacle	TFmini filter, warning, hold, fault behavior.	Controlled obstacle test passed.
P11	Payload bench	Servo + latch, AUX1, human authorization state machine.	Bench release passed.
P12	Integrated flight	AI + telemetry + search + event + obstacle + RTH in controlled flights.	Mission acceptance tests passed.
P13	Digital twin	Gazebo + SITL + replay + same event schema.	Repeatable simulation scenarios.
P14	Hardening	Spare parts, boot scripts, log extraction, rehearsals, documentation.	Demo-ready release.

14.1 Strict build order

POWER -> PROPULSION -> FLIGHT -> COMPANION -> AI -> MAVLink -> GCS -> EVENTS -> GEO -> SEARCH -> OBSTACLE -> PAYLOAD -> INTEGRATED MISSION -> DIGITAL TWIN -> DEMO HARDENING

DO NOT REVERSE SAFETY ORDER: Do not attach the payload or begin autonomous search flights before stable manual flight, GPS, RTH and battery failsafe tests are complete.

15. Hardware Build Procedure

15.1 Step-by-step mechanical assembly

1. Inspect the F450 frame plates, arms, screws and landing legs. Confirm no cracked carbon/plastic or stripped threads.
2. Mount the four motors at the four arm ends. Mark the physical positions permanently as M1 front-right, M2 rear-left, M3 front-left, M4 rear-right.

3. Mount the ESCs close enough to the motors to keep phase wires short but far enough from the Pixhawk compass area. Secure them against vibration and chafing.
4. Mount the PDB centrally and keep the high-current battery wiring short and mechanically supported.
5. Mount Pixhawk on vibration isolation near the center of gravity. Align the arrow/orientation exactly with the frame forward direction selected in software.
6. Mount the GPS/compass on a raised mast with clear sky view and physical separation from the battery/PDB/ESC wiring.
7. Mount the Raspberry Pi + AI HAT+ on a protected plate with cooling airflow; keep the camera ribbon cable radius gentle and protected.
8. Mount the Camera Module 3 on a rigid repeatable bracket near-nadir or at the selected fixed tilt. Record the physical pitch/roll/yaw offset.
9. Mount TFmini Plus parallel to the intended forward body direction. Ensure the propeller disc does not occlude its beam.
10. Mount the ELRS receiver antenna in a safe location away from high-current wires and carbon shadowing as far as practical.
11. Mount SiK telemetry air radio and route its antenna according to the radio manufacturer guidance.
12. Mount the payload latch low and centrally enough that it does not shift the center of gravity beyond the flight-test envelope.
13. Secure all battery straps, electronics fasteners and cable paths. Add strain relief to every cable crossing a moving/vibrating section.

15.2 Electrical bench sequence

1. With no battery connected, use a multimeter to confirm no short between battery positive and ground.
2. Connect the charger to the battery only according to the battery/charger manual and verify cell count and balance charging.
3. Connect the battery to the PDB through the XT60 and verify the PDB output voltage before connecting sensitive electronics.
4. Connect the power module to Pixhawk and confirm the Pixhawk powers without the motors enabled.
5. Configure and measure the dedicated Pi buck output at 5.1V under no-load and dummy-load conditions. Do not exceed the Pi input limit.
6. Configure and measure the servo BEC at 5V under a dummy load before attaching the servo.
7. Power the Pi and verify stable boot, AI HAT detection and camera detection.
8. Power the Pixhawk and verify GPS lock, compass status and pre-arm health flags.
9. Only after the above pass should the ESC signal wires be connected for motor dry testing.

15.3 Motor dry-test procedure

1. Remove all propellers.
2. Connect each ESC signal and signal ground to its assigned Pixhawk output.
3. Run motor test from Mission Planner at the minimum level needed to identify each motor.
4. Confirm M1/M2/M3/M4 physical positions match the corrected diagram.
5. Confirm each motor rotates in the required direction.
6. If an individual brushless motor direction is wrong, correct it at the ESC phase wiring or approved ESC direction setting, then repeat the test.
7. Perform an arming/failsafe test with no props and an emergency disarm action.

15.4 First flight procedure

1. Fly only in the approved controlled test area and only under the event/site safety rules that govern the prototype.
2. Use a manual/assisted mode first. Do not start with an autonomous mission.
3. Confirm stable hover, yaw control, pitch/roll response, GPS position hold and landing.
4. Test RTL at a low-risk condition while maintaining pilot readiness.
5. Log the flight and inspect EKF/GPS/battery data before the next flight.
6. Repeat until the flight controller baseline is stable. Only then add the Pi AI stack to the integrated flight.

16. Software Build and Integration Procedure

16.1 Pi OS and AI HAT setup

1. Install a current 64-bit Raspberry Pi OS release validated for Pi 5 AI workflows and record the exact release/build identifier.
2. Install system updates, then install the Raspberry Pi AI HAT software packages appropriate to the Hailo-8L device.
3. Confirm the accelerator is detected with the Hailo command-line identification tool.
4. Connect Camera Module 3 before completing the HAT installation sequence, following the current Raspberry Pi hardware order.
5. Install Picamera2/rpicam, OpenCV and the Python environment.
6. Run a camera-only preview and capture test before running AI.
7. Run a simple Hailo-compatible object-detection sample before loading the custom model.

16.2 Model training and export

1. Create the class taxonomy: person, vehicle, debris.
2. Collect license-compatible images and annotate bounding boxes.
3. Split the dataset into train/validation/test at the scene level.
4. Train YOLO11n on the development computer.
5. Evaluate precision, recall, mAP50 and mAP50-95 and inspect failure examples.
6. Export to Hailo using the Hailo target matching Hailo-8L and a fixed 640x640 input shape.
7. Use a representative calibration dataset and record the DFC/compiler version.
8. Copy the complete HEF export directory including metadata to the Pi.
9. Benchmark inference latency/FPS and thermal behavior before flight.

16.3 MAVLink integration

1. Connect Pixhawk TELEM2 to the 3.3V USB-UART adapter, then the adapter to the Pi.
2. Open the serial device with pymavlink and wait for heartbeat.
3. Request a minimal telemetry set and build a local vehicle-state cache.
4. Add command wrappers for GUIDED/mission interactions but keep a strict allow-list of commands.
5. Implement link-loss detection and a watchdog that stops sending commands if MAVLink becomes stale.
6. Test command handling in SITL before using the physical aircraft.

16.4 Event manager

1. Normalize detector output to a common bounding-box schema.

2. Pass candidate detections into ByteTrack.
3. Confirm tracks using the 3-of-5 rule.
4. Run the geo-tag calculation against the latest synchronized pose.
5. Calculate priority score and assign the incident ID.
6. Write to SQLite/JSONL and publish to the dashboard.
7. Apply duplicate suppression by time and spatial cell.
8. Expose the event to the mission manager without directly releasing the payload.

16.5 Dashboard

The dashboard should be local-first. Use FastAPI for the local API, WebSocket for event/telemetry pushes, Leaflet/OpenStreetMap for mapping, and a simple MJPEG or RTP/WebRTC video path selected during integration. The dashboard is not the flight controller; it is an operator interface and an authorization source.

17. Integration, Calibration and Validation Gates

Build and Validation Gates

No higher-level capability is accepted until lower-level gates pass

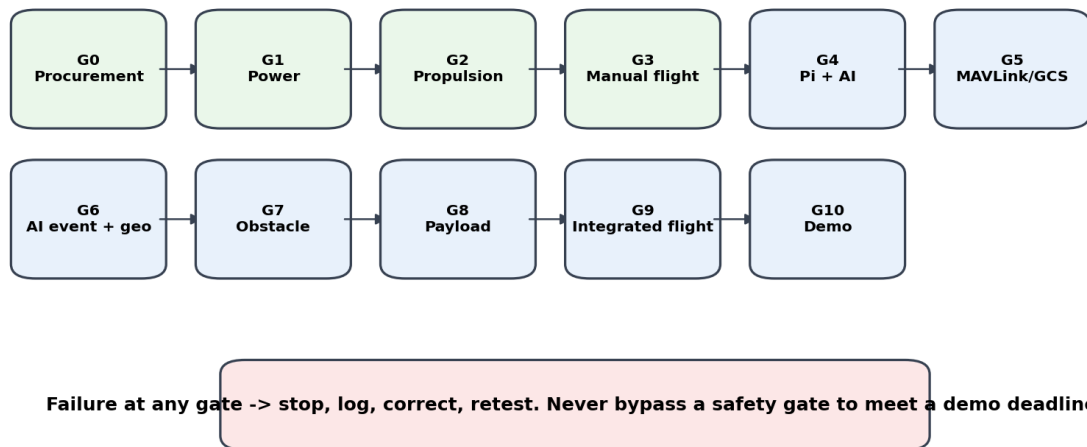


Figure 10. Ordered validation gates.

Gate	Area	Pass criterion
V1	Power	All rails measured at nominal and loaded conditions; no polarity error.
V2	Motor mapping	All motors on correct outputs and rotations; no propellers during dry test.
V3	Manual flight	Stable hover, control response, GPS, landing and logs.
V4	Failsafe	RC loss, battery low/critical, GCS/telemetry loss and GPS/EKF safety behavior tested.
V5	AI bench	Camera, Hailo and model inference are stable for >=30 minutes without thermal failure.
V6	MAVLink	Heartbeat, telemetry and allowed command path reliable in SITL and bench test.
V7	Geo-tag	Surveyed target coordinates produce measured error within the chosen acceptance band.
V8	Search	Generated lawnmower route stays inside the polygon and returns to home as configured.

Gate	Area	Pass criterion
V9	Obstacle	TFmini detects controlled test obstacles and produces warning/hold behavior.
V10	Payload	Servo release passes bench tests; flight payload allowed only after thrust margin.
V11	Integrated mission	Search -> detection -> alert -> response -> return/resume passes without safety faults.
V12	Demo repeatability	At least three representative end-to-end runs are logged with no critical regression.

17.1 ArduPilot battery failsafe baseline

ArduPilot documents a two-layer battery failsafe. The default low-voltage threshold is 10.5V, a 20% capacity threshold is a common starting point, and a reasonable action pair is low battery -> RTL and critical -> Land [W11]. For this prototype, use an initial parameter set such as: low 10.8V / 1040mAh -> RTL; critical 10.5V / 520mAh -> Land, then retune from measured battery sag and the exact cell chemistry. The parameter ordering must always satisfy low > critical [W12].

Parameter	Prototype initial value	Action / note
BATT_LOW_VOLT	10.8 V	Initial low-battery test threshold; validate sag.
BATT_LOW_MAH	1040 mAh	20% of 5200mAh planning capacity.
BATT_FS_LOW_ACT	2 = RTL	Low battery returns to home.
BATT_CRT_VOLT	10.5 V	Critical low-voltage test threshold.
BATT_CRT_MAH	520 mAh	10% of planning capacity.
BATT_FS_CRT_ACT	1 = Land	Critical battery forces landing.
BATT_FS_VOLTSRC	1 = sag compensated, if supported/validated	Avoid nuisance triggers from load sag; verify in actual firmware.

PARAMETER RULE: These are initial test values, not a substitute for the battery manufacturer safe operating limits or the actual measured voltage-sag profile.

17.2 Geofence

Use an inclusion geofence for the controlled demonstration and configure the fence action to a safe response such as RTL, subject to the flight site. The exact radius and altitude are site-specific; a 50m-radius, 30m-height example is a test parameter, not a statement of regulatory permission [W13].

18. End-to-End Working of the Built Prototype

This section describes what a successfully built AERIS does from power-on to mission completion. It is the operational story that should be used for demonstrations and for writing test scripts.

1. Battery is connected. The power module reports voltage/current to Pixhawk while the PDB feeds the propulsion branch and the dedicated regulators feed Pi and servo rails.
2. Pixhawk boots ArduPilot and runs pre-arm checks. The Pi boots Raspberry Pi OS, starts the watchdog, connects the camera, detects the Hailo NPU, initializes TFmini and opens the MAVLink connection.
3. The pilot confirms the aircraft has GPS/EKF/RC/battery health and arms using the normal ArduPilot procedure.
4. The operator selects a polygon search area or target coordinate on the ground station/dashboard.
5. The mission manager validates the route, battery reserve, geofence and vehicle state. If checks fail, the mission remains in PRECHECK.
6. Pixhawk performs the takeoff and follows the first waypoint. The Pi only supervises the mission context and reads telemetry.
7. During SEARCH, the Camera Module 3 streams frames. The Hailo-8L runs the compiled YOLO11n detector on a 640x640 representation.
8. Detections are associated into tracks with ByteTrack. A candidate becomes an event only after the temporal confirmation rule is met.

9. The event manager synchronizes the detection timestamp with the nearest valid vehicle telemetry sample and projects the detected image point onto the ground plane using the calibrated camera model and aircraft pose.
10. The incident is assigned confidence, track persistence, class severity and mission-context priority. It is written to SQLite/JSONL and sent to the ground dashboard.
11. The operator reviews the event. The drone can continue its search, or the operator can authorize a response mission to the incident coordinate.
12. During RESPONSE the Pixhawk controls flight. The Pi watches the mission state, battery reserve, obstacle state and telemetry freshness.
13. If TFmini detects an obstacle inside the stop band, the mission manager requests a pause/hold and the operator can decide whether to continue, reroute or abort.
14. At the target, the aircraft reaches a configured response state. Payload release is not automatic. The operator explicitly authorizes release after all interlocks pass.
15. The Pi sends the release command through MAVLink. Pixhawk actuates AUX1/Servo9 through the gripper function and the servo releases the small demonstration package.
16. After the response action, the mission manager requests RETURN or RESUME. Pixhawk continues to own the actual flight control.
17. At any time, critical battery, RC loss, geofence, GPS/EKF or other failsafes can override the mission. The buzzer/LED state provides local indication.
18. At mission completion, the system lands/returns according to the configured plan, flushes logs, stores the AI event list and preserves the flight log for post-flight analysis.

19. Digital Twin / SITL Implementation

The digital twin is not the physical F450 in disguise. It is a separate 5-kg-class hexacopter simulation target used to test mission software, route generation, event handling and response workflow before or alongside physical flights. The architecture intentionally keeps the perception and mission-manager interfaces the same while changing the simulated vehicle and dynamics model [D2][D5].

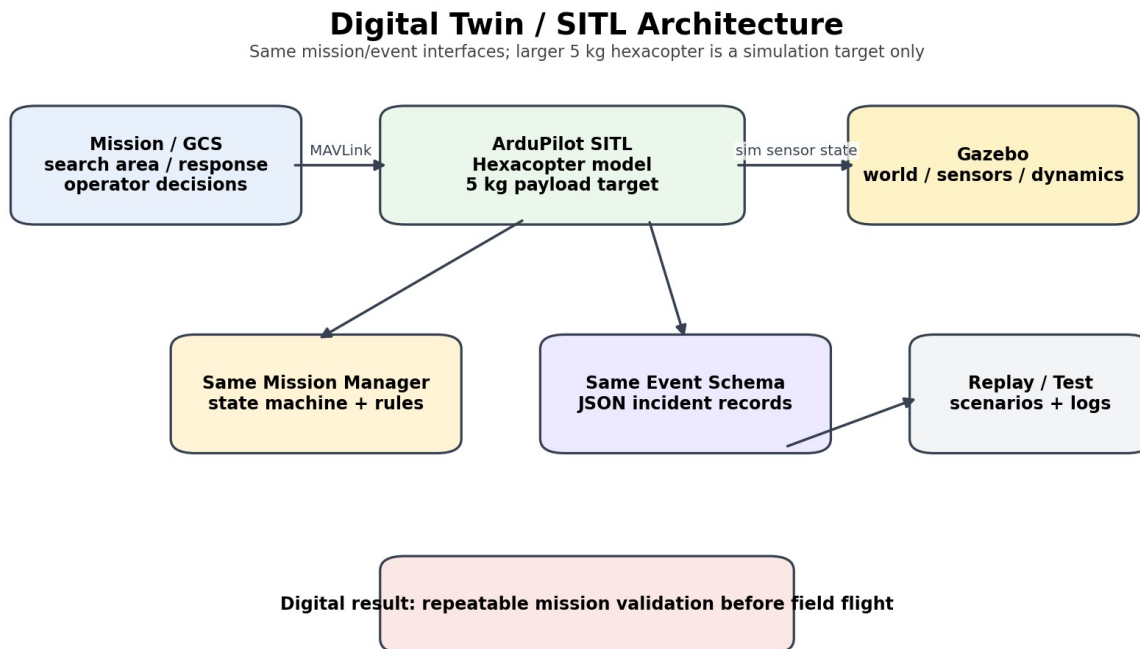


Figure 8. Digital-twin architecture.

19.1 SITL components

- ArduPilot SITL for the simulated flight controller.
- Gazebo world for vehicle, terrain and sensor simulation.
- MAVLink bridge for mission manager interaction.
- Simulated camera or recorded-video injector to exercise the detection pipeline.
- Simulated range sensor messages for obstacle tests.
- The same JSON incident schema used by physical flights.
- Replay scripts that feed known sequences of detections and telemetry to the state machine.

19.2 Simulation test scenarios

Scenario	Purpose	Pass condition
S01 normal search	Verify route execution	Coverage polygon completed.
S02 person detection	Verify event creation	One incident after confirmation; no duplicates.
S03 low-confidence detection	Verify human review	No autonomous response.
S04 obstacle	Verify pause/hold	Mission pauses and logs reason.
S05 battery low	Verify failsafe response	RTL/LAND state according to configured action.
S06 response	Verify target mission	Target waypoint reached and release gate shown.
S07 return/resume	Verify mission continuity	Search resumes from known context.
S08 communications loss	Verify safety	Flight controller owns configured failsafe.

20. Test Cases, Metrics, Logging and Acceptance

20.1 Hardware test matrix

ID	Test	Condition	Where	Pass
HW-001	Power polarity	No-load	Bench	No short; correct 5.1V/5V rails
HW-002	Pi rail load	Load at 25%/50%/100% test	Bench	Stable voltage; no brownout
HW-003	Motor mapping	Props removed	Bench	M1-M4 correct position/direction
HW-004	ESC thermal	5-minute staged run	Bench	No excessive heating or desync
HW-005	GPS/compass	Open sky	Bench	Healthy fix and compass status
HW-006	ELRS	Range/override test per site	Ground	Manual override reliable
HW-007	SiK	Bench/field link	Ground	Telemetry stable
HW-008	TFmini	Known distances	Ground	Range within expected tolerance
HW-009	Payload	Bench	Bench	Latch release reliable
HW-010	Flight	Controlled flight	Field	Stable hover and RTL

20.2 Software metrics

Metric	How measured	Target / interpretation
Inference latency	Timestamp frame before/after Hailo inference	Record p50/p95; low enough for search-speed choice.
Inference FPS	Processed frames / second	15-20 FPS target if achievable on release model.
Detection precision/recall	Held-out test set	Report actual measured values; no fixed claim before training.
Event false-alert rate	Controlled search runs	Minimize repeated/incorrect incidents; report count per run.
Geo error	Surveyed targets	Report median and 95th percentile in meters.
Mission route error	Compare commanded vs actual	No waypoint outside configured tolerance except safety diversion.
Telemetry freshness	Heartbeat/state timestamps	Trigger stale-state handling when timeout exceeded.

Metric	How measured	Target / interpretation
Pi temperature	CPU/thermal data	No thermal throttling during 30-minute bench AI soak.
Log completeness	Compare expected vs logged events	Every flight gets flight log + incident log + software release ID.

20.3 Event schema

```
{
  "mission_id": "M-2026-001",
  "event_id": "INC-0001",
  "timestamp_utc": "...",
  "class": "person",
  "confidence": 0.87,
  "track_id": 12,
  "drone_lat": 0.0,
  "drone_lon": 0.0,
  "target_lat": 0.0,
  "target_lon": 0.0,
  "altitude_m": 8.0,
  "priority": "HIGH",
  "geo_quality": "APPROXIMATE",
  "flight_mode": "AUTO",
  "operator_status": "UNVERIFIED"
}
```

20.4 Logging rules

- Use UTC timestamps for all event records; convert only at the UI.
- Keep the Pixhawk flight log and companion logs as separate source-of-truth artifacts but include a common mission_id.
- Flush the incident log after every confirmed event and again at mission completion.
- Log sensor health changes, not just successful samples.
- Never overwrite a previous flight log; use immutable mission IDs.
- Log exact software/config/model release identifiers in the mission header.

21. Calculations and Engineering Checks

21.1 Weight budget template

Mass item	Planning mass	Measured mass required
F450 frame + landing gear	300 g placeholder	Weigh actual
4x motors	200-240 g class placeholder	Weigh actual
4x ESCs	100-140 g class placeholder	Weigh actual
Propellers	20-30 g	Weigh actual
Pixhawk + GPS + wiring	100-140 g	Weigh actual
Pi 5 + HAT + cooler + SD + wiring	120-170 g	Weigh actual
Battery	400-500 g depending on pack	Weigh actual
Camera + TFmini + mounts	50-100 g	Weigh actual
Telemetry + ELRS	20-40 g	Weigh actual
Payload mechanism	30-60 g	Weigh actual
Reserve	100 g	Keep margin

MASS GATE: Do not approve the payload using the placeholder mass table. The final all-up weight must be measured with the exact production wiring, camera mount, battery and payload installed.

21.2 Center of gravity check

```
CG_x = sum(m_i*x_i) / sum(m_i)
CG_y = sum(m_i*y_i) / sum(m_i)
CG_z = sum(m_i*z_i) / sum(m_i)
```

The target is a center of gravity near the frame center and near the propeller plane used in the flight-controller tuning configuration. The battery is the primary mass-balancing component and should be mounted so that small payload changes do not shift the CG excessively.

21.3 Flight-time measurement method

1. Charge battery to the correct full voltage according to the battery/charger manual.
2. Zero/reset flight log and record initial voltage/current/remaining capacity.
3. Fly a repeatable hover or mission profile at the selected test altitude.
4. Log average electrical power and motor throttle.
5. End the test at the configured low-battery failsafe threshold or earlier as required by the safety plan.
6. Calculate measured mission duration and energy used.
7. Repeat with payload and without payload to estimate the impact of the release mechanism.

21.4 Search-area timing examples

Search area	Altitude	Lane spacing	Approx straight-line distance	Approx straight-line time at 3m/s
30 x 30 m	8 m	8.3 m	~120 m plus turns	~40 s plus turns
50 x 50 m	8 m	8.3 m	~350 m plus turns	~117 s plus turns
50 x 50 m	10 m	10.4 m	~260 m plus turns	~87 s plus turns

The mission should always include takeoff, transit, turn, response and return reserve; therefore these sweep-only estimates are not the mission flight-time budget.

22. Failure Modes and Recovery Logic

ID	Failure	Detection	Action	System consequence
F-01	Battery low	Voltage/capacity threshold	RTL	Do not release; log event
F-02	Battery critical	Critical threshold	LAND	Abort all AI response actions
F-03	RC loss	ArduPilot RC failsafe	Configured RTL/LAND	Pilot link health is independent
F-04	GCS/SiK loss	Telemetry heartbeat timeout	Flight controller continues	No payload release
F-05	Pi crash	Watchdog/heartbeat missing	Pixhawk continues	AI unavailable; mission abort/return according to policy
F-06	Hailo failure	No inference heartbeat	Mission can continue only as configured	No false "AI healthy" status
F-07	Camera disconnected	Capture health failure	Event pipeline disabled	Flight remains controlled
F-08	TFmini invalid	Range invalid/stale	Conservative HOLD when needed	Operator can inspect
F-09	GPS/EKF unhealthy	Pre-arm/flight health	No autonomous response	Use pilot-controlled safety
F-10	Wrong motor direction	Dry test mismatch	Do not arm for flight	Correct wiring first
F-11	Servo jam	Release command no expected state	Abort release / return	Never repeat release blindly
F-12	Duplicate AI alerts	Track/dedup failure	Suppress event storm	Reprocess logs
F-13	Geo-tag invalid	Ray/pose geometry invalid	Fallback to aircraft position labeled approximate	No high-confidence target navigation based on invalid geometry
F-14	Wi-Fi video loss	Video heartbeat timeout	Continue telemetry/flight	Operator relies on GCS/SiK or aborts
F-15	Thermal throttle	Pi temperature threshold	Reduce AI load / abort mission	Land/return if required

23. Deployment and Demonstration Procedure

23.1 Pre-demo checklist

- Battery charged and physically inspected; no swelling or damage.
- Propellers correct, undamaged and installed in the documented rotation pattern.
- Pixhawk firmware/version/parameters match the release manifest.

- GPS lock and EKF state healthy.
- ELRS transmitter/receiver link healthy and manual override tested.
- SiK telemetry healthy.
- Pi boot complete; AI HAT detected; camera detected; TFmini detected.
- AI model hash matches the release artifact.
- Mission polygon validated inside geofence.
- Battery reserve sufficient for search + return + contingency.
- Payload empty/secured until the payload gate is explicitly passed.
- Operator and spotter roles assigned.

23.2 Recommended demo sequence

1. Show the architecture and explain the safety split: Pixhawk flies; Pi interprets.
2. Show the live ground station with telemetry and local video.
3. Arm and perform a short manual/assisted takeoff and hover.
4. Execute a small autonomous search polygon.
5. Allow the detector to generate a person/vehicle/debris event in a controlled scenario.
6. Show the event on the map with confidence, timestamp and approximate coordinates.
7. Demonstrate the operator verification step.
8. If payload has passed flight validation, demonstrate a small release; otherwise show the gripper on the bench and keep the flight mission non-payload.
9. Complete RTL and show the post-flight event/flight logs.

24. Known Limits and Remaining Measured Quantities

The architecture is frozen, but some quantities cannot honestly be made exact in a document before the real hardware is assembled. These are not missing architecture; they are empirical quantities that must be measured.

Quantity	Why it remains empirical
Exact all-up mass	Depends on purchased board, wiring, mounts, battery lot and payload latch.
Actual motor thrust	Depends on exact motor/prop/battery combination and motor condition.
Actual hover power	Depends on mass, prop balance, battery voltage and flight tune.
Actual flight time	Must be measured under the selected mission profile.
Actual AI precision/recall	Depends on the trained dataset/model and deployment conditions.
Actual Hailo FPS/latency	Depends on exact HEF, runtime and pipeline configuration.
Actual geo-tag error	Depends on camera calibration, GNSS environment, attitude and ground-plane assumptions.
Actual obstacle stopping distance	Depends on airspeed, inertia, controller response and surface conditions.
Actual radio range	Depends on legal transmit power, antennas, RF environment and site.
Exact Pixhawk connector pinout	Must be verified on the purchased board/clone revision.

ENGINEERING STATUS: These are the correct remaining unknowns. The system does not need a different architecture to resolve them; it needs measurements and calibration runs.

25. Final Validation and Summary

25.1 Final engineering validation

Area	Status	Comment
Concept	PASS	Clear disaster-response mission and societal application.

Area	Status	Comment
Physical architecture	PASS WITH GATES	F450 + Pixhawk + Pi + Hailo + camera + TFmini is coherent; build must pass thrust/power tests.
Power architecture	FIXED	Dedicated 5.1V >=5A Pi rail and dedicated servo BEC added.
Motor mapping	FIXED	Standard Quad-X map explicitly frozen.
Communication	FIXED	SiK for MAVLink, Wi-Fi for video/events, ELRS for manual override.
AI pipeline	FIXED	YOLO11n + Hailo-8L + ByteTrack + event manager.
Geo-tagging	FIXED	Ground-plane projection with approximate fallback.
Obstacle behavior	FIXED	Forward sensing; warning/hold first; no 360-degree claim.
Mission logic	FIXED	Explicit state machine and fail-safe precedence.
Payload	FIXED WITH VALIDATION GATE	Human-authorized only; small payload only after measured flight margin.
Digital twin	PASS	Gazebo + ArduPilot SITL with shared mission/event interfaces.
Documentation	PASS FOR BUILD START	Exact pin map, BOM, software stack, procedures and test gates are defined here. Board-specific pin verification remains mandatory.

25.2 The complete prototype in one statement

When built to this specification, AERIS will be a small F450 autonomous quadcopter that can be assigned a controlled search area or target coordinate, autonomously navigate under ArduPilot, capture live camera data, perform edge object detection on the Raspberry Pi AI HAT+ using a custom YOLO11n model, track and confirm detections, associate them with an approximate target location using GPS/pose/camera geometry, classify them by transparent rules, alert a ground operator, use forward TFmini sensing for conservative obstacle-aware behavior, accept human-supervised response commands, optionally release a small validated demonstration payload through an ArduPilot gripper output, and return or resume the mission while maintaining independent manual override and flight-controller failsafes.

25.3 What is already complete in this document

- Idea, mission, applications, use cases and capability boundary.
- Final hardware BOM, quantities, technical roles and planning prices.
- Power topology, thrust/energy calculations and weight/thrust checks.
- Full software stack, AI model choice, data strategy, mathematical methods and algorithms.
- End-to-end hardware/software architecture and ownership boundaries.
- Electrical interface plan, pin-by-pin connection tables and data-flow diagrams.
- Full implementation order for hardware and software, including small runtime processes.
- Operational explanation of the fully built prototype.
- Digital-twin/SITL design, test cases, metrics, FMEA and demo procedure.
- Known empirical quantities and the exact remaining measurements needed before final claims.

25.4 Remaining checklist before procurement is irreversible

- Confirm exact Pixhawk board manufacturer/revision and compare its connector pinout with this specification.
- Confirm the selected RP2/ELRS receiver firmware and CRSF wiring.
- Confirm the selected SiK telemetry band is legal/appropriate for the deployment location and event.
- Confirm the exact buck converter and servo BEC models meet the current/thermal specifications.
- Confirm the exact battery model and its safe current/charging limits.

- Finalize the CAD mounts and verify center of gravity with the real parts.
- Create the final parameter file from the exact ArduPilot firmware release and perform a pre-arm audit.
- Create the actual trained YOLO11n model and Hailo HEF; update the release manifest with hashes.
- Run the hardware acceptance gates before attempting the integrated mission.

FINAL STATUS: The architecture is ready to build. The remaining items are component-lot verification, configuration freeze and measurement gates - not a redesign of the system.

26. Technical References

The following references were used for the refinement layer. The supplied AERIS documents remain the primary source for the project concept, prototype scope and original component intent; official vendor/flight-controller documentation was used to resolve interfaces and current capabilities.

Ref	Source	Location
[D1]	AERIS - AI-Powered Autonomous Disaster-Response UAV Concept & Prototype Document	Supplied project document, 7 pages.
[D2]	AERIS / GUJCOST Technical Details for Hackathon	Supplied project document, hardware/software tables and assembly diagrams.
[D3]	AERIS GUJCOST Robofest Technical Application Final	Supplied project application, 8 pages.
[D4]	R620260083 All Details	Supplied Robofest application record.
[D5]	AI-Assisted Autonomous Disaster Response UAV - Final Hackathon Build Plan, Revision 1	Supplied project document, 19 pages.
[W1]	Raspberry Pi Documentation - AI HATs	https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html
[W2]	Raspberry Pi - AI HAT+ product page	https://www.raspberrypi.com/products/ai-hat/
[W3]	Raspberry Pi - Camera Module 3	https://www.raspberrypi.com/products/camera-module-3/
[W4]	Raspberry Pi - Getting Started / Power	https://www.raspberrypi.com/documentation/computers/getting-started.html
[W5]	Raspberry Pi Documentation - AI software	https://www.raspberrypi.com/documentation/computers/ai.html
[W6]	ArduPilot - Quad/Output mapping documentation	https://ardupilot.org/copter/docs/common-rcoutput-mapping.html
[W7]	ArduPilot - Servo Gripper	https://ardupilot.org/copter/docs/common-gripper-servo.html
[W8]	ArduPilot - Battery Failsafe	https://ardupilot.org/copter/docs/failsafe-battery.html
[W9]	ArduPilot - Pre-Arm Safety Checks	https://ardupilot.org/copter/docs/common-prearm-safety-checks.html
[W10]	Ultralytics - Hailo export	https://docs.ultralytics.com/integrations/hailo
[W11]	Benewake - TFmini Plus product/manual family	https://www.benewake.com/TFminiPlus/index.html
[R1]	Redmon et al., You Only Look Once: Unified, Real-Time Object Detection	https://arxiv.org/abs/1506.02640
[R2]	Zhang et al., ByteTrack: Multi-Object Tracking by Associating Every Detection Box	https://arxiv.org/abs/2110.06864
[R3]	Galceran and Carreras, A survey on coverage path planning for robotics	https://www.sciencedirect.com/science/article/pii/S092188901300167X
[R4]	Kumar and Kumar, Region coverage-aware path planning for unmanned aerial vehicles: A systematic review	https://www.sciencedirect.com/science/article/pii/S1874490723000769

REFERENCE USE: Technical specifications in this document are deliberately phrased as planning values, official vendor properties, or measured quantities to avoid converting a component datasheet into a guarantee about the final assembled aircraft.

Appendix A - Master Configuration Snapshot

Parameter	Frozen baseline
Frame	F450, Quad-X
Motor mapping	M1 FR CCW / M2 RL CCW / M3 FL CW / M4 RR CW
Motor	A2212 1000KV x4
ESC	30A x4

Parameter	Frozen baseline
Prop	1045, 2 CW + 2 CCW
Battery	3S 11.1V 5200mAh
Flight controller	Pixhawk 2.4.8 / FMUv2-class
Firmware	ArduCopter stable release, exact version frozen at build
GPS	M8N-class GPS + compass
Companion	Raspberry Pi 5 4GB
AI	AI HAT+ Hailo-8L 13 TOPS
Camera	Camera Module 3 Standard
Range	TFmini Plus
RC	RadioMaster Pocket ELRS + RP2
Telemetry	SiK pair
Pi power	5.1V >=5A dedicated buck
Servo power	5V 3A dedicated BEC
MAVLink Pi link	TELEM2 -> 3.3V USB-UART -> Pi
ELRS link	SERIAL4 CRSF
Telemetry link	TELEM1 MAVLink
Payload	AUX1/Servo9 gripper; human-authorized
AI classes	person / vehicle / debris
AI input	640x640
AI confidence baseline	0.35 candidate / 0.60 event
Tracking	ByteTrack
Event confirm	3 of 5 frames
Event cooldown	10 s + spatial suppression
Search altitude	8-10m AGL nominal controlled demo; tune from detection tests
Search speed	2.5-3.0 m/s baseline
Overlap	20%
Obstacle warning	8.0m
Obstacle hold	6.5m initial
Obstacle critical	5.5m initial/future refinement
Battery low	10.8V / 1040mAh -> RTL initial test values
Battery critical	10.5V / 520mAh -> Land initial test values
Geofence	Site-specific; example 50m radius / 30m height for controlled demo
Physical payload	Small demo only after thrust and flight validation
5kg payload	Digital-twin target only

Appendix B - Final Go/No-Go Checklist

Check	YES/NO	Verified by	Date
Exact Pixhawk board/revision verified			
Power rails measured under load			
Motor map and rotations verified			
Props installed correctly			
GPS/compass calibrated			
RC override verified			
SiK telemetry verified			
Pi/Hailo/camera healthy			
TFmini valid			
YOLO HEF hash verified			
MAVLink heartbeat stable			
Mission route validated			
Geo-tag calibration measured			
Battery failsafe tested			
Payload bench tested			
Integrated mission repeated 3 times			
Logs archived			

END OF SPECIFICATION: This master document is the reference build baseline. Any later hardware, pin, model, parameter or mission-logic change should create a new revision rather than being silently changed in the workshop.